

BMI Calculator with Gender-Aware Health Advisory

NAME = SHOURYA CHATURVEDI

REGISTRATION NO. = 26BCE11364

**COURSE NAME : INTRODUCTION TO PROBLEM
SOLVING AND PROGRAMMING**

COURSE CODE : CSE1021

FACULTY NAME : VENKAT PRASAD PADHY

DATE OF SUBMISSION : 28 SEPTEMBER 2026

1. Introduction

Body Mass Index has been the go-to quick-screening number for weight status for decades — it's cheap to compute, needs no special equipment, and correlates reasonably well with body fat at a population level. The catch is that a raw BMI number on its own doesn't mean much to the person reading it. Being told your BMI is "27.3" is not useful; being told that puts you in the Overweight band, what that tends to mean for someone of your gender, and what you can practically do about it — that's useful.

This project takes the standard BMI formula and wraps it in exactly that layer of context. It is a Python command-line application that accepts a person's basic details and body measurements in either metric or imperial units, computes BMI, classifies it into one of five recognised health bands, and returns gender-specific consequences and practical cures for that band. It also supports processing several people back-to-back in one sitting and finishes with a consolidated, colour-coded summary table — a small but genuinely useful feature for anyone running group screenings, such as a trainer, a school health coordinator, or a small clinic.

The rest of this report walks through the problem this solves, what the system is and isn't meant to do, how it's put together internally, the design decisions behind it, how it was tested, and what was learned along the way.

2. Problem Statement

Free BMI calculators are everywhere online, but almost all of them stop at a number and a one-word label. They rarely explain what that label means for the specific person reading it, and they almost never account for the fact that identical BMI values carry different health implications for men and women — for instance, hormonal and bone-density risks that show up disproportionately in underweight women, or cardiovascular strain patterns that present differently by gender at the same BMI. On top of that, nearly every calculator is built strictly for one person at a time, which becomes tedious the moment someone needs to screen a group — a family, a classroom, a gym's new intake batch — and ends up manually re-typing results into a spreadsheet afterwards.

This project addresses both gaps directly: accurate BMI computation from either unit system, gender-aware categorisation with real, actionable advice, and native support for multi-person sessions that end in one clean summary table — all from a single, dependency-free Python script.

3. Functional Requirements

The functional requirements describe what the system must be able to do. They were derived directly from the problem statement above and map one-to-one onto the program's actual behaviour:

- **FR-1:** Accept a person's name, age, and gender (restricted to *men* or *women*, since the health-advisory data is structured around these two categories).
- **FR-2:** Accept body weight in either kilograms or pounds, converting pounds to kilograms internally before any calculation.

- **FR-3:** Accept height in either centimetres or inches, converting both to metres internally before any calculation.
- **FR-4:** Calculate BMI using the standard formula, weight (kg) divided by height (m) squared.
- **FR-5:** Classify the computed BMI into exactly one of five bands — Underweight, Healthy Weight, Overweight, Obese Category I, Obese Category II.
- **FR-6:** Return gender-specific consequences and a gender-specific cure/recommendation for the identified band.
- **FR-7:** Display each person's result immediately, colour-coded by category severity.
- **FR-8:** Allow the user to add another person and repeat the whole flow, without restarting the program.
- **FR-9:** Once the session ends, print one consolidated table listing every person entered, with their BMI and category still colour-coded.

Input / output structure: all input is collected interactively through the console using Python's `input()`, and all output is printed directly to the console using ANSI-coloured `print()` statements — there is no file or network I/O in this version, keeping the tool simple to run anywhere Python is installed.

4. Non-Functional Requirements

Beyond what the system does, these describe the qualities it needs to have to be genuinely pleasant and dependable to use:

Quality	How it's addressed in this project
Usability	Menu-style prompts, colour-coded severity, and plain-English advice mean no technical jargon
Reliability	Gender, unit, and height inputs are explicitly validated; invalid input raises a clear error message
Maintainability	Each responsibility — calculation, categorisation, reporting, control flow — is in its own module
Performance	BMI computation and lookup are both $O(1)$ per person; even a session with dozens of people is fast
Resource efficiency	Pure standard-library Python with one tiny local helper module — no heavy dependencies
Error-handling strategy	Invalid gender, missing height, or an unrecognised weight unit each raise a specific, readable error

5. System Architecture

The system follows a simple layered structure even though it lives in a single file. Input from the user flows into a session controller, which delegates the actual maths to a calculation engine and the classification/advice lookup to a categorisation module, uses a small colour-constants module for presentation, and finally hands collected records to a reporting module for the end-of-session table. Nothing is persisted beyond the running process — all records live in a plain Python list of dictionaries for the duration of the session.

Figure 1 — System Architecture

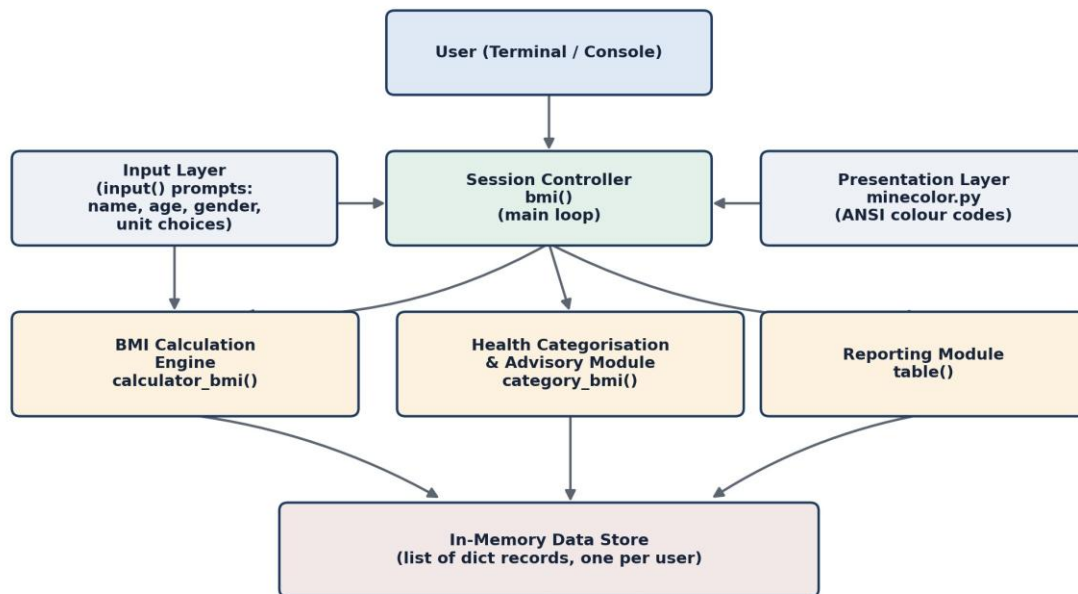


Figure 1 — System architecture: how input, calculation, categorisation, presentation, and reporting layers connect.

6. Design Diagrams

6.1 Process Flow / Workflow Diagram

This traces exactly what happens from the moment the program starts to the moment it ends, including the loop that lets a user add multiple people before the final table is printed.

Figure 2 — Process Flow / Workflow

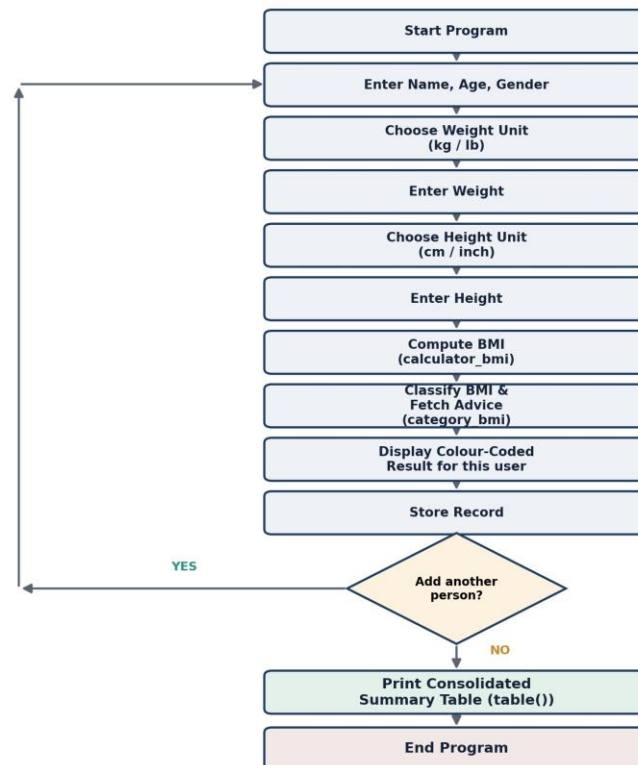


Figure 2 — End-to-end process flow, including the repeat-for-another-person loop.

6.2 Use Case Diagram

There is a single actor — the person operating the console — who directly triggers three primary use cases (entering details, choosing units, and deciding whether to add another person). The remaining use cases — calculating BMI, viewing the category and advice, and viewing the summary table — happen automatically as a consequence of those, which is why they appear inside the system boundary without a direct line from the actor.

Figure 3 — Use Case Diagram

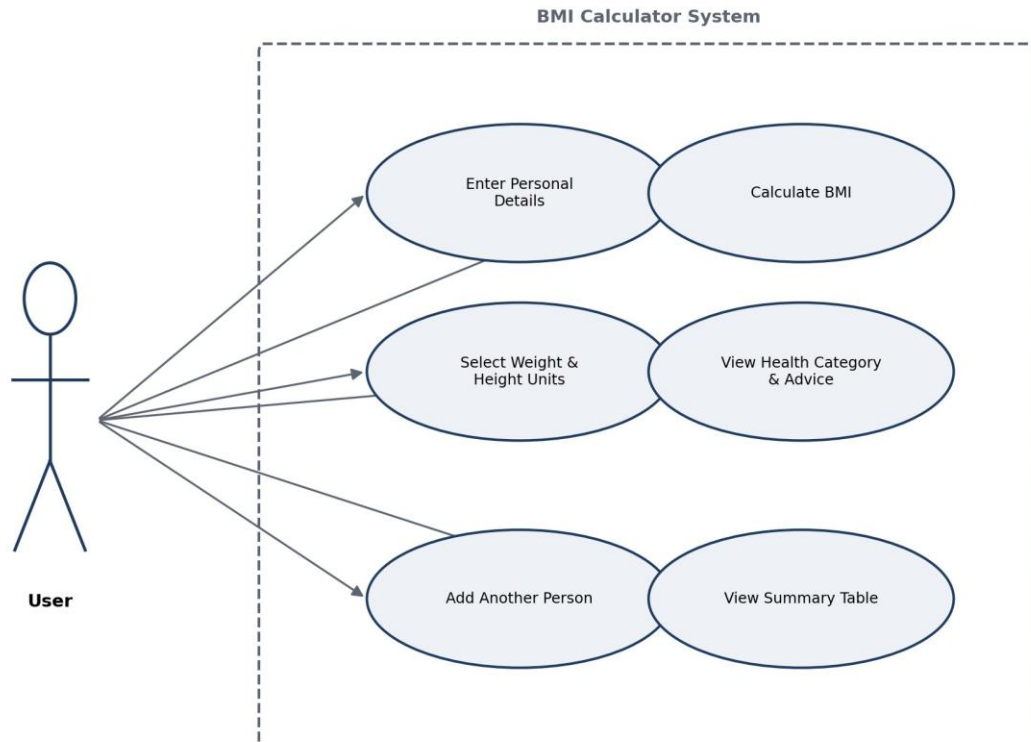


Figure 3 — Use case diagram for the BMI Calculator system.

6.3 Class / Component Diagram

Since the project is written as a set of focused functions rather than formal classes, this diagram treats each function as a component with its own inputs, internal responsibility, and outputs — which is really what a class diagram is trying to communicate anyway: clear boundaries of responsibility.

Figure 4 — Class / Component Diagram

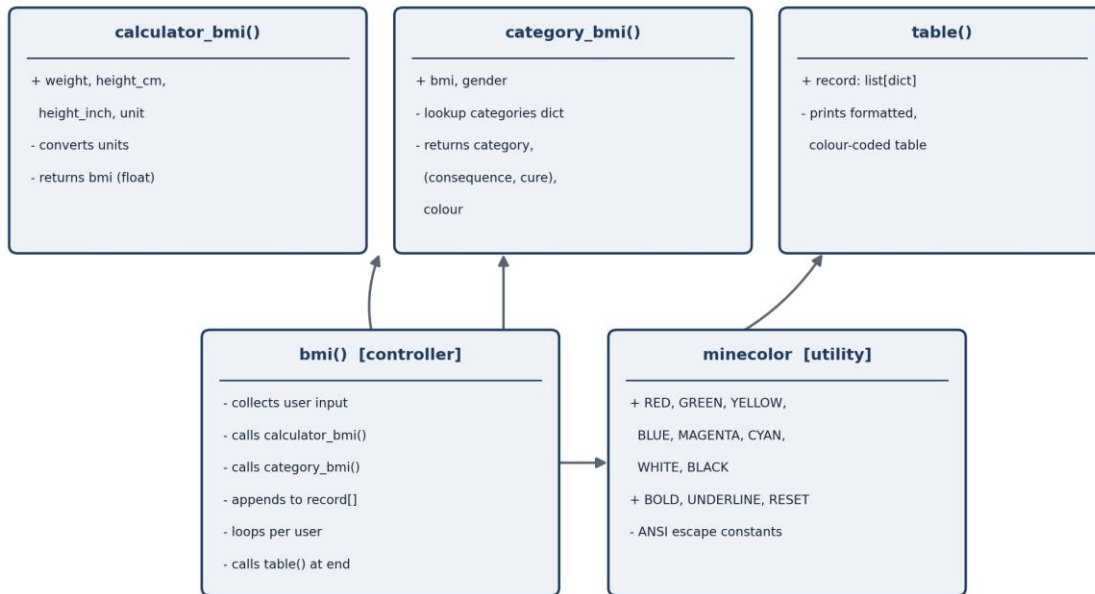


Figure 4 — Component view of the four functional modules plus the colour-constants utility.

6.4 Sequence Diagram

This shows a single pass through the system for one person: the controller collects input, calls the calculation engine, then the categorisation module, displays the result, stores the record, and — once the loop eventually ends — hands every stored record to the reporting module in one final call.

Figure 5 — Sequence Diagram (single user pass)

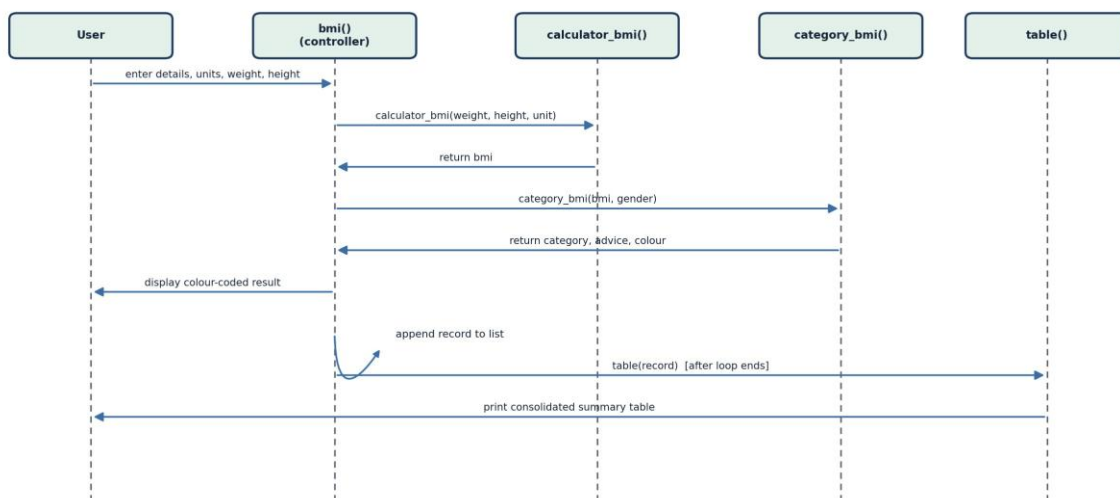


Figure 5 — Sequence of calls for a single user pass through the system.

6.5 Database / Storage Design

This project intentionally has **no persistent storage** — there is no file, database, or external service involved, so a formal ER diagram doesn't apply here. What does exist is a simple in-memory record schema: every person is stored as one Python dictionary, and all dictionaries for the session live in a single list. The shape of that dictionary is shown below for completeness, since it's effectively the project's only “data model”.

Field	Type	Description
name	str	Person's entered name
age	int	Person's entered age
gender	str	'Men' or 'Women' (capitalised for display)
bmi	float	Computed BMI value
category	str	One of the five BMI bands
consequences	str	Gender-specific health consequences for that band
cure	str	Gender-specific recommendation for that band

7. Design Decisions & Rationale

Why a single script instead of a package of modules? For a CLI tool of this size, splitting four small functions across four separate files would add navigation overhead without adding real clarity. The functions are already separated by responsibility — calculation, categorisation, reporting, control — so the modularity exists at the function level even though the file boundary is shared. Section 12 (Future Enhancements) revisits this as the project scales.

Why a dictionary of dictionaries for category data instead of a class per category? The category data (consequences and cures per gender per band) is pure content, not behaviour — it never needs methods of its own. A nested dictionary keeps that content in one place, easy for a non-programmer (say, a health advisor reviewing the wording) to read and edit without touching any logic.

Why convert units internally rather than branching the whole calculation per unit? Converting weight to kilograms and height to metres immediately means the core BMI formula only ever has to be written once, in one place, which removes an entire class of unit-mismatch bugs.

Why colour-code by category instead of just printing plain text? Colour gives an instant visual read on severity — green versus red communicates far faster than reading every word, especially when scanning a table of several people at once.

Why a plain in-memory list instead of a file or database? The project is scoped as a single-session screening tool, not a records system. Adding storage would have meant adding schema design, file handling, and error handling for a feature outside the stated problem — better handled as a deliberate future enhancement than bolted on half-finished.

8. Implementation Details

The program is written in plain Python 3 with no third-party dependencies. It's organised into four functions plus one small local helper module, as already covered in the architecture and component diagrams above. A short walkthrough of the two most important functions follows.

8.1 `calculator_bmi()`

Accepts weight, an optional height in centimetres, an optional height in inches, and a unit flag for weight. Pounds are converted to kilograms with the standard factor 0.453592; inches are converted to metres with the standard factor 0.0254. If neither height option is supplied, or the weight unit is neither 'kg' nor 'lb', the function raises a `ValueError` rather than silently guessing.

```
bmi = weight_in_kg / (height_in_metres ** 2)
```

8.2 `category_bmi()`

Takes the computed BMI and the person's gender, checks the BMI against the five standard clinical thresholds (18.5, 25, 30, 35), and returns the category name, the gender-specific (consequences, cure) tuple from the nested dictionary, and the colour to use for that category — all in one call, keeping the calling code in `bmi()` short and readable.

8.3 `table()` and `bmi()` [controller]

`table()` re-applies the same colour logic to every stored record and prints a fixed-width, aligned table using Python's f-string formatting (`f"{value:<15}"`) so columns line up regardless of content length. `bmi()` is the entry point: it drives the whole interactive loop, calling the other three functions in sequence for each person and only calling `table()` once, after the loop naturally ends.

9. Screenshots / Results

The screenshot below is a genuine terminal session, not a mock-up — it was captured by actually running `sc_bmi_calculator.py` with two sample people, **Aarav** (24, male, 72 kg, 175 cm) and **Priya** (29, female, 58 kg, 160 cm). Both land in the Healthy Weight band, shown consistently in green in both their individual results and the final summary table.

```

bash - tc_bmi_calculator.py

***** BMI CALCULATOR *****
ENTER YOUR NAME: Aarav
ENTER YOUR AGE: 24
ENTER YOUR GENDER (men/women): men
CHOOSE WEIGHT UNIT
1- KILOGRAMS
2- POUNDS
ENTER YOUR CHOICE (1 or 2): 1
ENTER YOUR WEIGHT: 72
CHOOSE HEIGHT UNIT
1- CENTIMETERS
2- INCHES
ENTER YOUR CHOICE (1 or 2): 1
ENTER YOUR HEIGHT IN CENTIMETERS: 175

***** RESULT *****
NAME: Aarav
AGE: 24
GENDER: men
YOUR BMI IS: 23.51
CATEGORY: HEALTHY WEIGHT
CONSEQUENCES: Strong Metabolism, Good Stamina and Physically Fit, Risk of disease is low
CURE: Maintain Balanced diet, Regular Exercise, Keep Hydrated Yourself

WANT TO ADD ANOTHER PERSON ? (YES/NO): YES
ENTER YOUR NAME: Priya
ENTER YOUR AGE: 29
ENTER YOUR GENDER (men/women): women
CHOOSE WEIGHT UNIT
1- KILOGRAMS
2- POUNDS
ENTER YOUR CHOICE (1 or 2): 1
ENTER YOUR WEIGHT: 8
CHOOSE HEIGHT UNIT
1- CENTIMETERS
2- INCHES
ENTER YOUR CHOICE (1 or 2): 1
ENTER YOUR HEIGHT IN CENTIMETERS: 160

***** RESULT *****
NAME: Priya
AGE: 29
GENDER: women
YOUR BMI IS: 22.66
CATEGORY: HEALTHY WEIGHT
CONSEQUENCES: Healthy Hormones, Strong bones, Good Fertility
CURE: Maintain Balanced Diet, Yoga, Physical Exercise

WANT TO ADD ANOTHER PERSON ? (YES/NO): NO

***** BMI TABLE *****
NAME    AGE    GENDER    BMI    CATEGORY    CONSEQUENCES    CURE
-----
Aarav   24     Men       23.51   HEALTHY WEIGHT   Strong Metabolism, Good Stamina and Physically Fit, Risk of disease is low   Maintain Balanced diet, Regular Exercise, Keep Hydrated Yourself
Priya   29     Women     22.66   HEALTHY WEIGHT   Healthy Hormones, Strong bones, Good Fertility   Maintain Balanced Diet, Yoga, Physical Exercise

```

Figure 6 — Real terminal output for a two-person session, ending in the consolidated summary table.

Aarav's BMI computes to 23.51 and Priya's to 22.66 — both correctly placed in the 18.5–25 Healthy Weight band, with the gender-specific advice for each visibly different (metabolism/stamina framing for Aarav, hormonal/bone-density framing for Priya), which is exactly the behaviour Section 3's functional requirements call for.

10. Testing Approach

This version of the project doesn't ship an automated test suite (see Section 12 for why that's flagged as a near-term improvement), so verification was done through structured manual test runs covering five categories:

- **Happy path:** normal, realistic inputs for both genders, confirming BMI, category, and colour all agree with each other.
- **Unit-conversion consistency:** the same real body size entered once in metric and once in imperial, confirming both runs land on the same BMI within rounding.
- **Boundary values:** inputs engineered to sit right at 18.5, 25, 30, and 35 to confirm each band's edge is classified correctly (the code uses inclusive lower bounds, e.g. exactly 18.5 counts as Healthy Weight, not Underweight).

- **Invalid input:** an unrecognised gender and an unrecognised unit choice, confirming the program raises a clear error and exits rather than crashing with a raw traceback or producing a silently wrong result.
- **Multi-person sessions:** three people entered back-to-back, confirming the final table lists every person with the correct data and colour, in entry order.

The worked example in Section 9 corresponds directly to the first and fifth of these categories, run together in a single session.

11. Challenges Faced

Keeping gender-specific advice genuinely different, not just relabelled. It would have been easy to write one generic piece of advice per BMI band and simply swap the word 'he' for 'she'. Actually researching and writing consequences and cures that reflect real physiological differences — like bone density and fertility considerations for women versus cardiovascular and prostate-adjacent risks for men — took noticeably more care than the calculation logic itself.

Getting column alignment right in the summary table. Mixing plain strings with ANSI colour codes in fixed-width formatting is a classic gotcha: colour codes are invisible characters that still count toward string length in naive formatting, which can silently break column alignment. The final implementation applies the colour escape codes around the already-padded string rather than before padding, which keeps columns lined up regardless of category.

Deciding where validation should live. Early drafts scattered input-checking across multiple functions. Consolidating gender and unit validation at clear, single points made the error behaviour far more predictable and easier to reason about.

12. Learnings & Key Takeaways

- A calculation is rarely the hard part of a “calculator” project — the BMI formula is one line; the value is entirely in how the result is classified, explained, and presented.
- Separating *data* (the category dictionary) from *logic* (the functions that use it) made the project noticeably easier to review, correct, and extend without touching working code.
- Small usability choices — colour-coding, aligned tables, clear error messages — make a disproportionate difference to how “finished” a simple CLI tool feels, for very little extra code.
- Terminal output that mixes ANSI escape codes with formatted strings needs careful handling, since invisible control characters can quietly break naive width-based formatting.

13. Future Enhancements

- **Split into a proper package** — separate files for the calculation engine, the categorisation/advisory data, the reporting layer, and the CLI controller, with a clear folder structure as the project grows.
- **Add an automated test suite** (e.g. with `pytest`) covering the boundary and invalid-input cases already exercised manually in Section 10.
- **Persist session data** to a CSV or lightweight SQLite file, so results survive beyond a single run and can be reviewed or exported later.
- **Support additional demographic granularity** (e.g. age-banded advice, since risk profiles shift with age even within the same gender and BMI band).
- **Optional graphical or web front-end** for non-technical users who'd rather not use a terminal at all, reusing the same calculation and categorisation functions underneath.

14. References

- World Health Organization — BMI classification and standard cut-off points for adults.
- Centers for Disease Control and Prevention (CDC) — Body Mass Index: Considerations for Practitioners.
- Python 3 official documentation — `string` formatting and standard library reference.
- ANSI escape code reference — standard terminal colour/formatting codes used by the `minicolor` helper module.

— End of Report —